Breachmap — Design Doc

Oct 26, 2026 · @kk

Summary

Breachmap finds how an attacker could chain small weaknesses across code, CI/CD, cloud and Kubernetes into a real breach, then proposes the cheapest fix that breaks every chain and proves the fix worked.

It reads your GitHub repos, workflows, AWS accounts, Kubernetes clusters and existing scanner results, and joins them into one graph of "who can reach what". A graph algorithm finds attack paths to your sensitive data and picks the smallest set of changes that cuts all of them. An AI writes the actual patch, but only the graph decides where to fix and only deterministic checks decide whether the fix is accepted.

Accuracy is the core feature, not an afterthought. Every link in a path carries its evidence, a confidence level and the time it was observed. Links are checked against AWS's own evaluators, gaps in visibility are reported instead of hidden, and accuracy is measured on labs where the right answer is known in advance.

The problem

Each security tool sees one layer, so no one sees the chain. A secret scanner flags a token in a repo, a cloud scanner flags a broad IAM role, a Kubernetes scanner flags an over-privileged service account. Each is filed as "medium" in a different dashboard, owned by a different team. An attacker doesn't care about layers; they walk from one weakness to the next.

A real, documented example: Tinder's security team found that many AWS roles trusted GitHub Actions through OIDC without checking the token's `sub` claim. That meant a workflow in *anyone's* repository could assume the role. They found vulnerable setups in multiple organizations within hours, including roles owned by AWS itself.

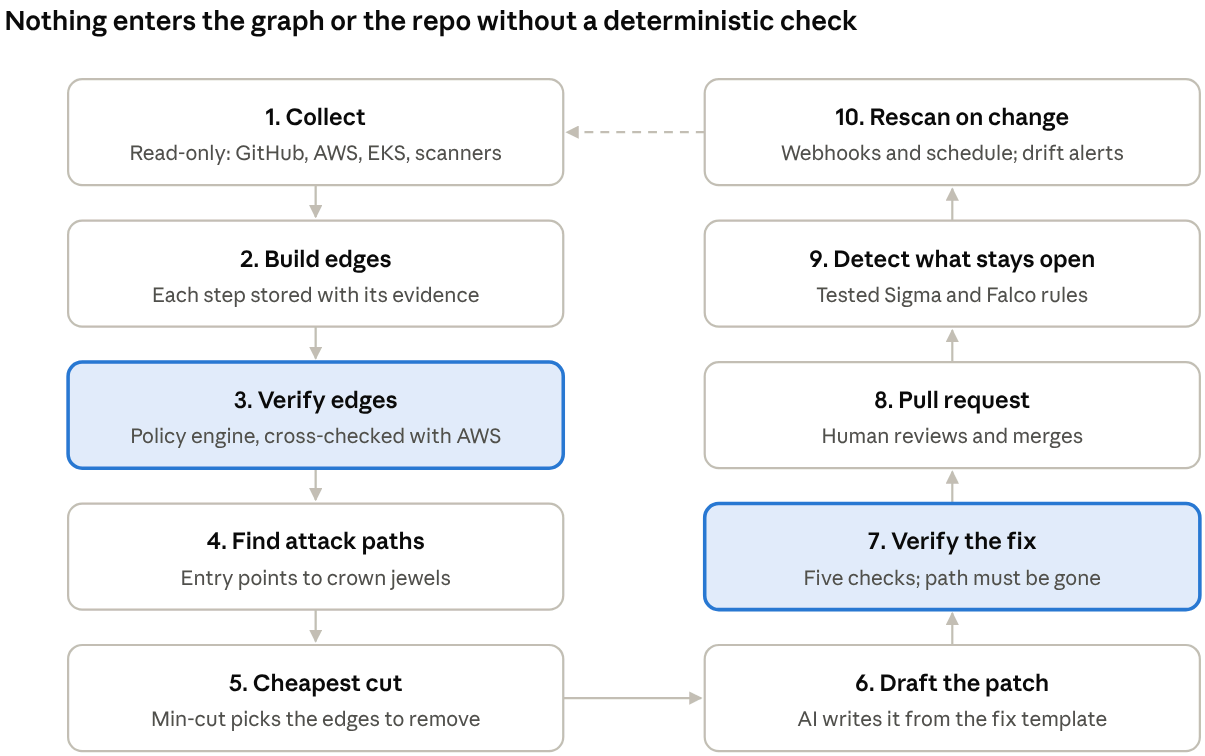
On its own, that trust policy looks like a small IAM mistake. Connected to what the role can reach (a deploy role, then a cluster, then a data bucket), it is a full breach from outside the company. That connection is what Breachmap exists to find.

What teams lack today:

- **One graph across layers.** Open-source tools like Cartography map assets into a graph, but don't compute attack paths or fixes across code, CI and cloud.
- **A fix that targets the choke point.** Scanners list hundreds of findings; nobody says "fix these two and every path to the database is gone."
- **Honest accuracy.** Tools present guesses as facts and don't say what they couldn't see.

How it works

Breachmap runs as a continuous loop. Steps 1 to 5 build the graph and choose the fix; steps 6 to 8 turn it into a verified pull request; steps 9 and 10 watch what is still open and catch new paths as things change.



Breachmap loop · 10 steps, 2 verification gates

The two highlighted boxes are where accuracy is enforced. An edge can't enter the graph without evidence and a verification result, and a patch can't become a PR unless the rebuilt graph proves the path is gone. The AI appears only at step 6.

Domain coverage

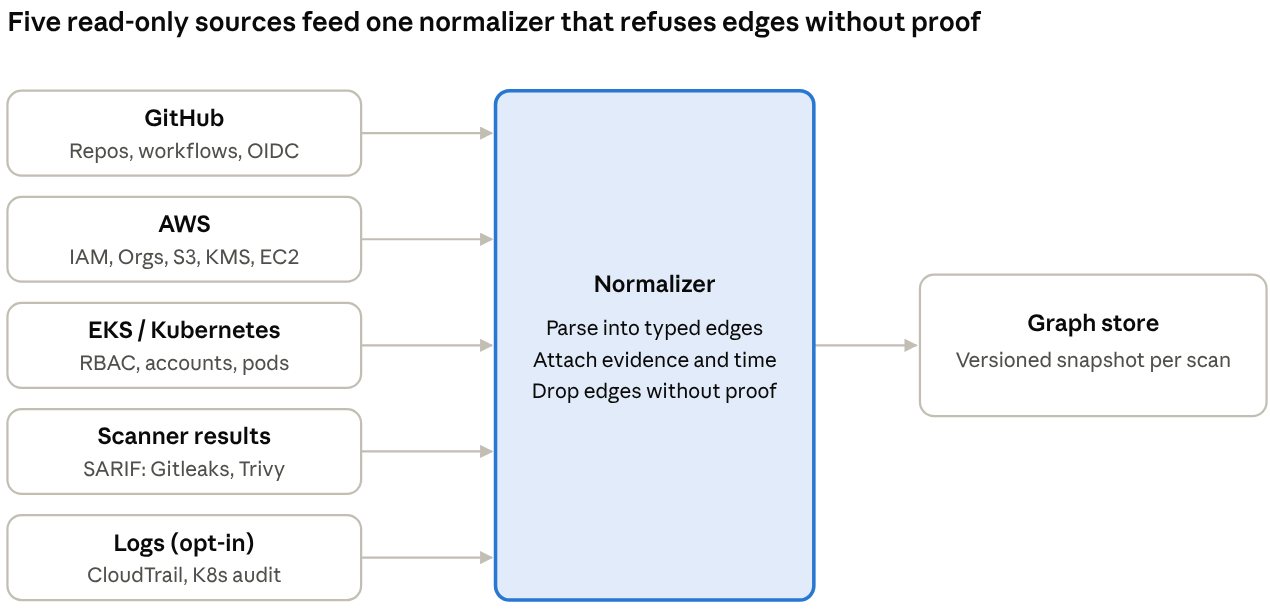
Every field gets a real, working piece of the system, not a buzzword. Each row is something a person from that field could review in an interview.

Field	What Breachmap reads	What it computes	What it outputs
Cloud (AWS)	IAM users, roles, policies, trust policies, SCPs, S3/KMS/Secrets Manager policies, security groups, public endpoints	Who can assume what, who can read what, what is reachable from the internet	Cloud attack paths, least-privilege IAM fixes as Terraform PRs
Kubernetes	RBAC roles and bindings, service accounts, pods, Services/Ingress, EKS role links (IRSA / Pod Identity)	Which pods can reach which cloud roles and which other workloads	RBAC and service-account fixes as manifest PRs
DevOps	GitHub Actions workflows, triggers, runners, environments, branch protection	Which workflows can be started by outsiders, which get cloud credentials	Workflow fixes as PRs
DevSecOps	Secrets (Gitleaks), vulnerable dependencies and images (Trivy, OSV), code findings (Semgrep) as SARIF	Which findings are actual entry points onto a path, versus noise	Prioritized findings: "fix this one, it leads to customer data"
Offensive security	Everything above as one graph	Attack paths, chained privilege escalation, MITRE ATT&CK technique per step	Path explorer, attack narrative, lab replay
SOC / detection	CloudTrail, Kubernetes audit logs (lab and opt-in prod)	Which log events would show each step of a path	Sigma and Falco rules per path, live alerts when a path becomes open

Field	What Breachmap reads	What it computes	What it outputs
GRC / compliance	Same graph	Which controls each path violates	CIS AWS / CIS Kubernetes / ISO 27001 mapping, exportable report
Platform engineering	n/a	n/a	Connector framework, job workers, graph DB, API, dashboard, Helm chart
AI	Code and config around a chosen fix	n/a, never used for scoring	Plain-English explanations and patch drafts, always verified

Data collection

Breachmap reads from five kinds of sources, all read-only, and reads **live state**, not just Terraform files, because live state is what an attacker actually faces. Every collector runs as a background job and records what it could and couldn't read.



Data collection · 5 sources into one normalizer

Source	Access Breachmap asks for	Why
GitHub	GitHub App with read access to contents, Actions, metadata; write only to open PRs	Workflows, triggers, OIDC usage, branch protection
AWS	Read-only role per account (security-audit style), assumed with short-lived credentials and an external ID; Organizations read from the management account	IAM, trust policies, SCPs, resource policies
EKS / Kubernetes	Read-only ClusterRole for RBAC, service accounts, pods, services	Who can run what, which pod gets which cloud role
Scanners	Upload or CI artifact in SARIF	Leaked secrets and vulnerable components as entry points
Logs	Opt-in CloudTrail and K8s audit access	Detection testing and "is this path being used"

Drift: when a Terraform file and live state disagree, live state wins for path-finding, and the disagreement is shown as its own finding.

Coverage report: every scan lists accounts, regions, clusters and repos it covered, plus any API calls that were denied. Paths through anything not covered are labeled *possible*, never silently dropped.

The graph model

Everything is a node, and every "can reach" relationship is a directed edge. An attack path is simply a route through edges from an **entry point** to a **crown jewel**.

Node types

Category	Nodes
Entry points	`Internet`, `AnyGitHubUser`, `PublicRepo`, `PublicEndpoint`, `LeakedSecret`, `VulnerableImage`

Category	Nodes
Code and CI	`Repo`, `Branch`, `Workflow`, `Job`, `Environment`, `GitHubOIDCClaim`
Cloud	`AwsAccount`, `IamRole`, `IamUser`, `Policy`, `S3Bucket`, `KmsKey`, `Secret`, `Ec2Instance`, `Lambda`, `SecurityGroup`
Kubernetes	`Cluster`, `Namespace`, `ServiceAccount`, `Pod`, `Role`, `ClusterRole`, `Service`, `Ingress`
Crown jewels	Any node the user tags as sensitive, e.g. a customer-data bucket or production database secret

Edge types (each one is a single attacker step)

Edge	Meaning	Example
`CAN_TRIGGER`	Actor can start a workflow	`AnyGitHubUser` → `Workflow` via `pull_request_target`
`GETS_TOKEN_FOR`	Workflow receives an OIDC token with these claims	`Workflow` → `GitHubOIDCClaim(repo:acme/app:ref:refs/heads/main)`
`CAN_ASSUME`	Principal can assume an IAM role	`GitHubOIDCClaim` → `IamRole` when the trust policy matches
`CAN_ACCESS`	Principal can perform a sensitive action on a resource	`IamRole` → `S3Bucket` (`s3:GetObject`)
`CAN_EXEC_INTO` / `CAN_CREATE_POD`	Kubernetes privilege to run code in a workload	`IamRole` → `Cluster` via EKS access entry

Edge	Meaning	Example
`RUNS_AS`	Workload uses a cloud identity	`Pod` → `ServiceAccount` → `IamRole` (IRSA)
`EXPOSED_TO`	Network reachability	`Internet` → `Service` (LoadBalancer)
`CONTAINS`	Asset holds a usable credential	`Repo` → `LeakedSecret` → `IamUser`

Every edge stores the same metadata: the **evidence** (the exact policy statement, file and line, or API response that proves it), the **evidence level**, the **confidence**, the **time observed**, the **collector version**, and a **fix cost** with the fix template that would remove it. Without that metadata an edge is not allowed into the graph.

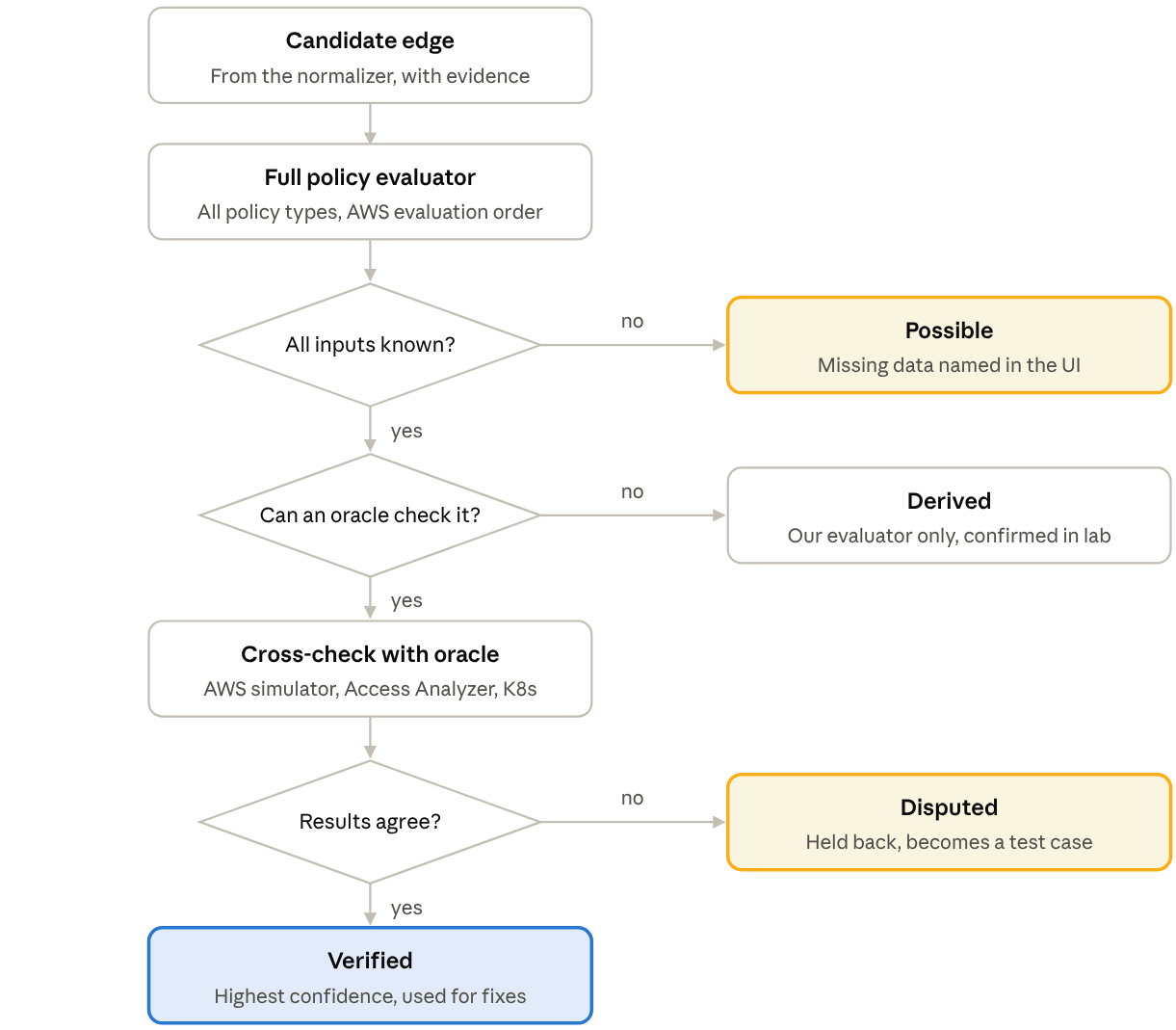
The accuracy engine

Accuracy comes from three rules: compute every permission with a complete evaluator, double-check it with an independent oracle where one exists, and never present a guess as a fact.

1. A complete policy evaluator. Breachmap implements AWS's documented policy evaluation logic for every request it models: explicit denies anywhere win; Organizations SCPs and resource control policies must allow; resource-based policies, permission boundaries, session policies and identity policies are combined in AWS's order; and conditions (source IP, org ID, tags, OIDC `sub` and `aud`) are evaluated, not ignored. Most tools check identity policies only, which is the biggest source of false positives.

2. Independent oracles. Where AWS or Kubernetes can answer the same question, Breachmap asks and compares:

- The IAM policy simulator API for a principal's actions. Its documented gaps (no resource control policies, resource policies must be supplied, possible differences with VPC endpoint policies or role chaining) are exactly why it's a cross-check and not the primary engine.
- IAM Access Analyzer, which uses automated reasoning over IAM policies to report external and internal access to resources.
- Kubernetes' own authorization API (`SubjectAccessReview`, what `kubectl auth can-i` uses) for RBAC edges.



Accuracy engine · how one edge earns its evidence level

3. Evidence levels, shown everywhere.

Level	Meaning	Used for
Verified	Our evaluator and an independent oracle agree	Paths, fixes, alerts, reports
Derived	No oracle exists for this edge type (e.g. GitHub trigger rules); computed deterministically and covered by lab tests	Paths and fixes, labeled

Level	Meaning	Used for
Possible	An input is missing: an unreadable account, a condition value we can't know	Shown separately, never auto-fixed
Disputed	Our evaluator and the oracle disagree	Held back, flagged, turned into a regression test

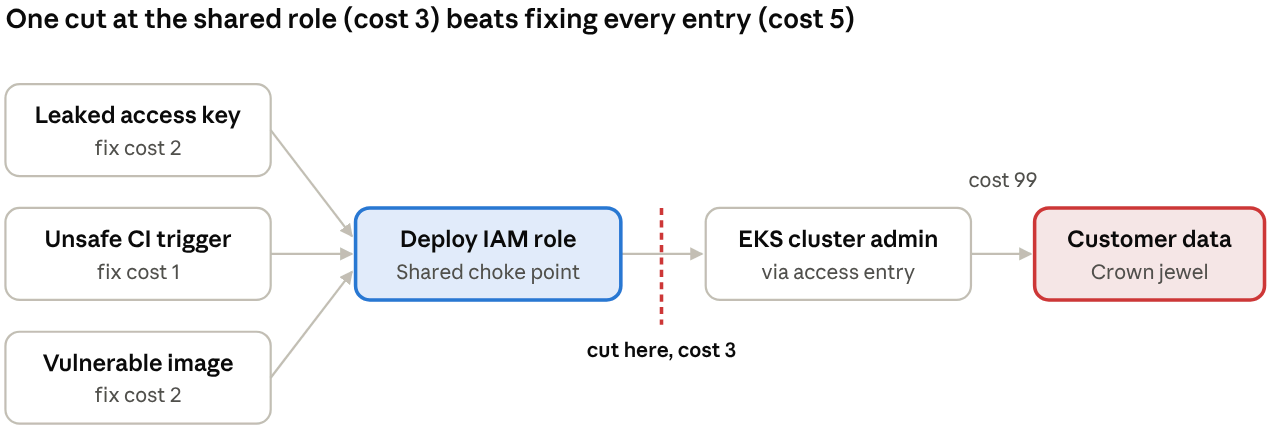
A path's level is the **lowest level of any edge on it**, the weakest link. Each edge also carries the time it was observed; edges older than the last successful scan are marked stale. The dashboard always shows how many paths are Verified versus Derived versus Possible, so nobody mistakes a guess for a confirmed breach route.

Path finding and the cheapest fix

The graph decides where to fix, using a standard algorithm, so the answer is reproducible and provable. "Cheapest" means least effort and least disruption, never lowest quality.

Finding paths. All entry points are joined to one virtual "attacker" node and all crown jewels to one virtual "target" node. Breachmap lists every path between them (bounded by length) and scores each by evidence level, number of steps and what it reaches.

Choosing the fix. Every edge has a fix cost from its fix template: how many lines change, whether it can cause downtime, how many workloads depend on it. Edges that can't be removed (an app genuinely needs to read its own bucket) get a very high cost. A **minimum cut** then finds the cheapest set of edges whose removal disconnects attacker from target, which covers every path at once.



Min-cut example · 3 entry points, 1 choke point

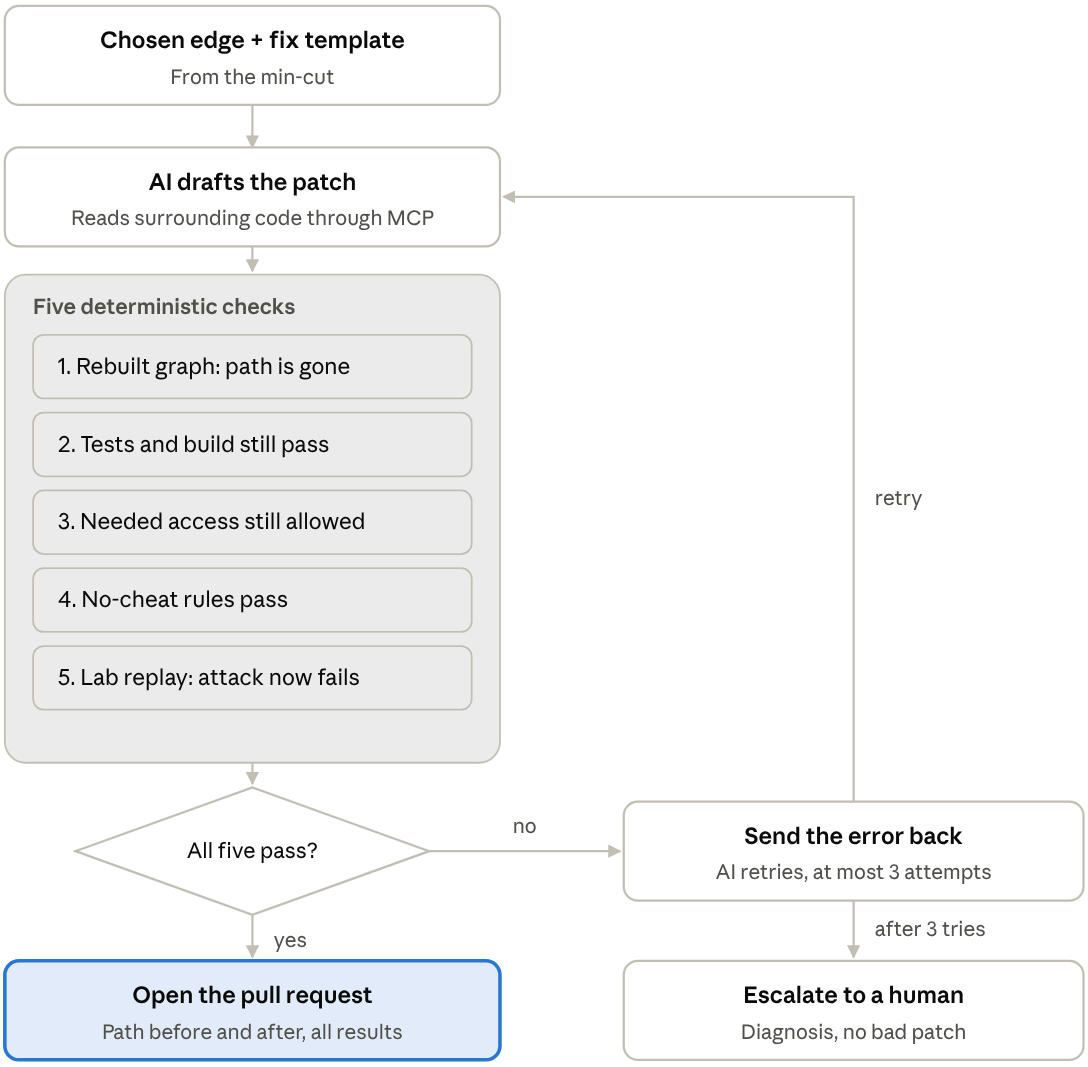
Fixing each entry point separately costs 2 + 1 + 2 = 5 and leaves the door open for the next new entry. Cutting the shared role's cluster access costs 3 and blocks all three paths, plus any future path through that role. Ties are broken by smaller blast radius, then by fewer files changed.

Only Verified and Derived edges are used for automatic fix plans. Paths that depend on a Possible edge get a separate "needs confirmation" plan that asks for the missing access first.

Fix generation and verification

The AI writes the patch; it never decides whether the patch is good. Each edge type has a **fix template** that states what must change (for example "restrict the OIDC trust condition to `repo:acme/app:ref:refs/heads/main`"). The AI uses MCP to read the actual Terraform, workflow or manifest and writes a minimal patch in the repo's own style.

An AI patch becomes a PR only after five deterministic checks pass



Fix verification · 5 checks, max 3 retries

What each check proves:

1. **Path is gone:** apply the patch to a copy of the graph inputs, re-run the evaluator, and confirm the cut edge and every path through it no longer exist.
2. **Nothing broke:** the repo's CI, `terraform plan`, `actionlint` and `kubectl --dry-run` all succeed.
3. **Needed access still works:** a list of legitimate actions for that role or workload (learned from usage logs or declared by the team) is re-checked and must still be allowed.
4. **No cheating:** the patch may not delete the resource, disable the workflow, add `*` permissions, add `continue-on-error`, or touch files outside the fix template's scope.

5. **Attack fails in the lab:** for lab accounts, the path's steps are actually replayed with harmless canary actions and must now be denied.

The PR shows the path before and after, the cost, every check result, and a one-line rollback.

Detection for the SOC

Not every path can be fixed today; some need a change window, an owner or a budget. For those, Breachmap turns the path into detections so the SOC sees it if someone actually walks it.

1. **One rule per step.** Each edge type has a detection template. A `CAN_ASSUME` edge from a GitHub OIDC claim becomes a Sigma rule on CloudTrail `AssumeRoleWithWebIdentity` events for that role whose subject doesn't match the expected repo. A `CAN_EXEC_INTO` edge becomes a Falco or Kubernetes-audit rule on `pods/exec` against that namespace.
2. **Path correlation.** A single step can be normal. Two or more steps of the same path within a time window raises a high-severity "path in progress" alert with the full path attached.
3. **Rules are tested.** Every generated rule ships with a sample log event that must trigger it and a normal event that must not. Rules that fail their own test are not exported.
4. **Path drift alerts.** When a new scan opens a path that was closed (someone changed a trust policy by hand), the SOC gets an alert immediately, with the diff that opened it.

Output formats: Sigma YAML (portable to most SIEMs), Falco rules, and a JSON webhook for Slack or a SOAR tool. In v1, log ingestion runs in the lab and on opt-in accounts only.

Compliance mapping

Auditors ask "which controls are failing and how bad is it"; Breachmap answers with paths instead of checklists. Each edge type maps to the controls it violates, so a path report doubles as compliance evidence.

Edge or condition	Framework area it maps to
Over-broad IAM trust or wildcard permissions	CIS AWS Foundations (IAM section); ISO 27001 access control; least privilege

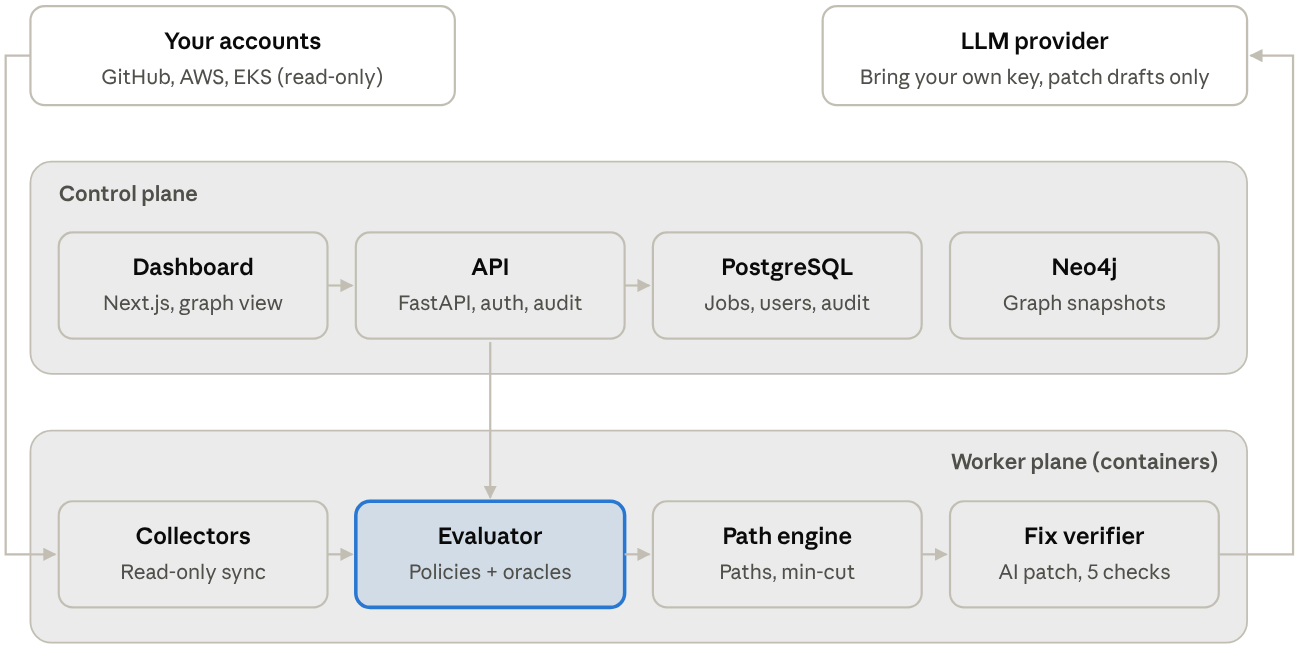
Edge or condition	Framework area it maps to
Unsafe CI trigger or unpinned third-party Action	Secure development and supply-chain controls; OpenSSF Scorecard checks
Leaked or long-lived credentials	Credential management and key rotation controls
Cluster-admin bindings, privileged pods	CIS Kubernetes Benchmark (RBAC and pod security sections)
Public exposure of sensitive data stores	Data protection and network segmentation controls

The exact control IDs come from a versioned mapping file that cites the benchmark version it was written against, so the report never claims a control ID it hasn't checked. Exports: PDF and CSV per account, with every finding linked back to its path and evidence.

Architecture and tech stack

The web app (control plane) holds users, jobs and results; all cloud access and heavy computation happen in workers (worker plane). Workers run as a pipeline: collect, evaluate, find paths, then draft and verify fixes.

Cloud credentials live only in workers; the web app never holds them



Architecture · control plane and worker plane

Layer	Choice	Why
Dashboard	Next.js + TypeScript, Cytoscape.js for the graph view	Interactive path explorer with evidence on click
API	FastAPI (Python)	Same language as the engine and as Concord; auto OpenAPI
Graph store	Neo4j	Path queries, compatible with Cartography's schema if we reuse its collectors
App database	PostgreSQL	Users, workspaces, jobs, audit log
Queue	Redis + RQ or Celery	Retries, dead-letter queue, scheduled rescans
Path engine	Python with NetworkX for min-cut on extracted subgraphs	Proven algorithms, easy to unit test

Layer	Choice	Why
Policy evaluator	Own Python implementation of AWS evaluation logic, plus boto3 calls to the simulator and Access Analyzer	Primary engine plus independent oracle
Kubernetes	Official Python client; `SubjectAccessReview` for verification	RBAC oracle
Detection	Sigma rules, Falco rules, tested with sample events	Portable to most SIEMs
Lab	Terraform for AWS + EKS + GitHub org, tagged and budget-capped	Known-answer accuracy tests
Packaging	Docker Compose for local, Helm chart for Kubernetes	Self-hosted, credentials stay with the user

Securing Breachmap itself

Breachmap holds a map of exactly how to breach its users, so it is a top-value target. Rule one: **it never needs write access to the cloud.** It reads everything and changes things only through pull requests a human merges.

Threat	Defense
Stolen Breachmap credentials give an attacker cloud access	Read-only cloud roles only (AWS `SecurityAudit`-style), assumed with short-lived credentials and an external ID; no long-lived keys stored
The attack-path database leaks	Paths are encrypted at rest; per-workspace isolation; path export requires an admin role and is audited
Prompt injection in repo code or tags ("ignore your rules")	Repo text is data only; the LLM never decides scores, edges or acceptance; secrets are redacted before any prompt
AI writes a malicious or fake fix	Five deterministic checks and a human merge (see Fix generation)

Threat	Defense
Lab replay hits a real environment by mistake	Replay runs only in accounts tagged `breachmap-lab=true` and on an allowlist; production is simulation-only, enforced in code
Forged GitHub webhooks	HMAC signature verification and delivery-ID deduplication
Tenant crossing	Workspace ID enforced on every query, server side

Every read, path computation, PR and export is written to an append-only audit log.

Proving accuracy

"Accurate" has to be a number we measure, not a claim. Breachmap is tested against environments where the right answer is known in advance, and every release must hit its targets before shipping.

Test environments (known answers)

1. **Seeded lab:** our own Terraform setup of AWS + EKS + GitHub with an answer key: the exact list of paths that should exist and the ones that look open but must not be reported (blocked by an SCP, a deny statement or a condition).
2. **CloudGoat:** Rhino Security Labs' vulnerable-by-design AWS scenarios, each with a documented intended path.
3. **Mutation tests:** take the seeded lab, change one thing (add a permission, remove a deny, widen a trust `sub`), and check that exactly the expected paths appear or disappear. Hundreds of mutations can be generated automatically.
4. **Adversarial cases:** deny statements, `NotAction`, wildcard ARNs, conditions on source IP or org ID, permission boundaries, cross-account trust, role chaining.

Metrics and v1 targets (targets are goals until measured)

Metric	Definition	v1 target
Path recall	Known paths found / all known paths	≥ 95%

Metric	Definition	v1 target
Path precision	Reported paths that are real / all reported paths	⩾ 95%
Edge agreement	Our IAM edges matching AWS's own evaluators (simulator, Access Analyzer)	⩾ 99% on verified edges
Mutation pass rate	Mutations producing exactly the expected path change	⩾ 95%
Fix success	Fix PRs that remove the path and keep tests and access working	⩾ 90%
Unknown rate	Edges marked unknown on the seeded lab	Reported, driven down release by release

Every false positive or miss found by a user becomes a new test fixture, so the same mistake can't come back.

Scope: v1 versus later

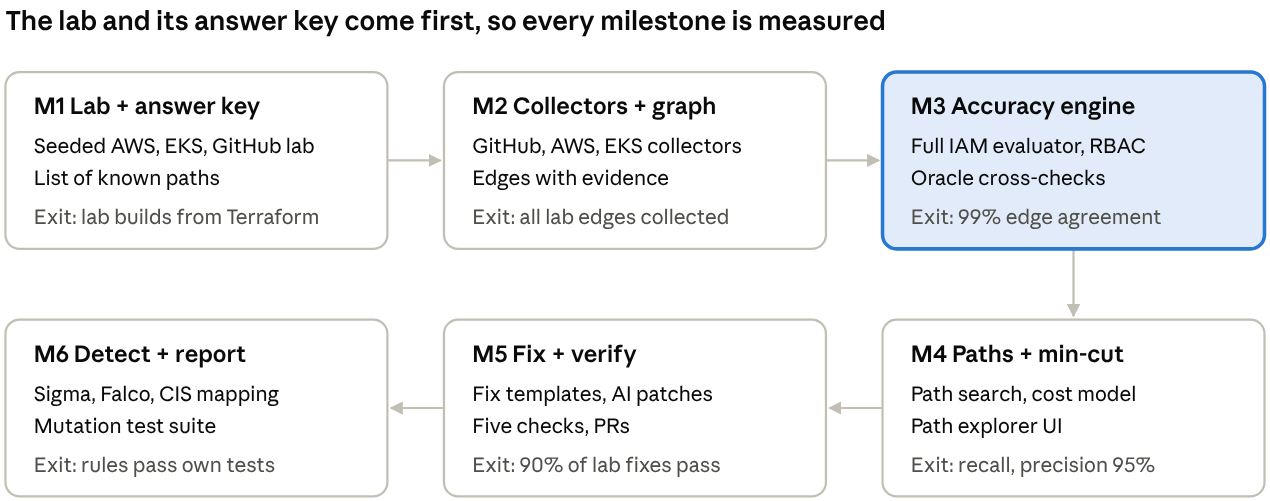
v1 covers one cloud, one CI and one Kubernetes flavor, done to the accuracy targets above. Breadth comes only after depth is proven.

Area	v1	Later
Cloud	AWS: IAM, STS, S3, KMS, Secrets Manager, EC2, Lambda, security groups, Organizations SCPs	Azure, GCP; RDS, DynamoDB; VPC endpoint policies
CI/CD	GitHub Actions + GitHub OIDC	GitLab CI, Jenkins
Kubernetes	EKS: RBAC, service accounts, IRSA / Pod Identity, access entries, Services	AKS, GKE; network policies; admission controllers
Scanner imports	SARIF from Gitleaks, Trivy, Semgrep, OSV-Scanner	Any SARIF or CycloneDX source

Area	v1	Later
Fixes	IAM trust and policy, workflow trigger and permissions, RBAC bindings	Network rules, multi-step refactors
Detection	Sigma (CloudTrail), Falco/K8s audit, tested	Native SIEM exports, SOAR playbooks
Compliance	CIS AWS + CIS Kubernetes mapping	ISO 27001, SOC 2 evidence packs
Replay	Lab accounts only	Never on production, by design

Build plan

The lab and its answer key are built first, so accuracy is measurable from day one instead of being tested at the end. Each milestone ships code, UI, tests and docs together and closes only when its exit test passes on the lab.



Build plan · 6 milestones with exit tests

M3 is highlighted because it is what makes Breachmap trustworthy; if time runs short, M1 to M4 already make a demoable, accurate attack-path tool.

Working with Claude Code and Codex: keep `CLAUDE.md` and `AGENTS.md` short (stack, commands, hard rules like "read-only cloud access" and "no edge without evidence"), keep this doc in `docs/`, and give each session one milestone file with its exit test. Every

IAM evaluation rule gets a unit test written from the AWS documentation before the code, so the agents can't quietly guess AWS behavior.

Open questions

- [ ] Build our own collectors or start from Cartography's AWS/K8s/GitHub ingestion and add edges on top? (Faster start vs full control of evidence metadata.)
- [ ] Graph store: Neo4j (Cartography-compatible) or Postgres with a NetworkX compute layer?
- [ ] Which AWS account hosts the seeded lab, and what is the monthly budget cap for it?
- [ ] Team split and owners per milestone.
- [ ] Does Concord become a data source (its triaged findings as entry points), or stay fully separate?
- [ ] Final name: Breachmap is a working title.

Sources

- AWS IAM policy evaluation logic
- IAM policy simulator guide, including its limitations
- IAM Access Analyzer findings
- Tinder: GitHub Actions and AWS OIDC misconfigurations
- Cartography (CNCF sandbox)
- CloudGoat by Rhino Security Labs